Explicación técnica completa

AGR — Práctica 2: redes, cumplimiento del enunciado, proceso y scripts

Pablo Elvira Calatayud

Índice

- 1. Cómo están conectadas las redes
  - 1.1. Parte 1: la VM de Vagrant en VirtualBox
  - 1.2. Parte 2: la réplica del laboratorio 3103 en KVM
- 2. Cumplimiento del enunciado, punto por punto
  - 2.1. Parte 1: “Virtualización de servicios web con Vagrant” (págs. 1–2)
  - 2.2. Parte 2: “Virtualización de infraestructura de red con KVM” (págs. 3–4)
  - 2.3. “Propuesta de solución” y “Recomendaciones” (pág. 4)
  - 2.4. Lo que falta o queda por confirmar
  - 2.5. Propuestas para reforzar el punto 2.17 (FRR)
- 3. Proceso completo: creación, integración, despliegue y testeo
  - 3.1. Parte 1 (Vagrant)
  - 3.2. Parte 2 (KVM)
- 4. Explicación detallada de cada script y fichero
  - 4.1. `parte1-vagrant/Vagrantfile` (Ruby)
  - 4.2. `parte2-kvm/agr_kvm.py` (Python 3)
  - 4.3. `parte2-kvm/topologia.json`
  - 4.4. `parte2-kvm/plantilla_vm.xml` (el XML de Moodle)
  - 4.5. `parte2-kvm/guest/build-base.sh` (se ejecuta una vez dentro de la imagen base)
  - 4.6. `parte2-kvm/guest/agr-net.sh` (en cada arranque de cada VM)
  - 4.7. `parte2-kvm/guest/agr-net.service`
  - 4.8. `parte2-kvm/guest/ssr-server.service`
  - 4.9. `parte2-kvm/guest/agr-info.sh` (al iniciar sesión)
  - 4.10. Ficheros generados por VM (ejemplo: RC)
  - 4.11. `doc/build_pdf.sh`
  - 4.12. El servidor web SSR-master-server (código de terceros)
- 5. Mapa del repositorio

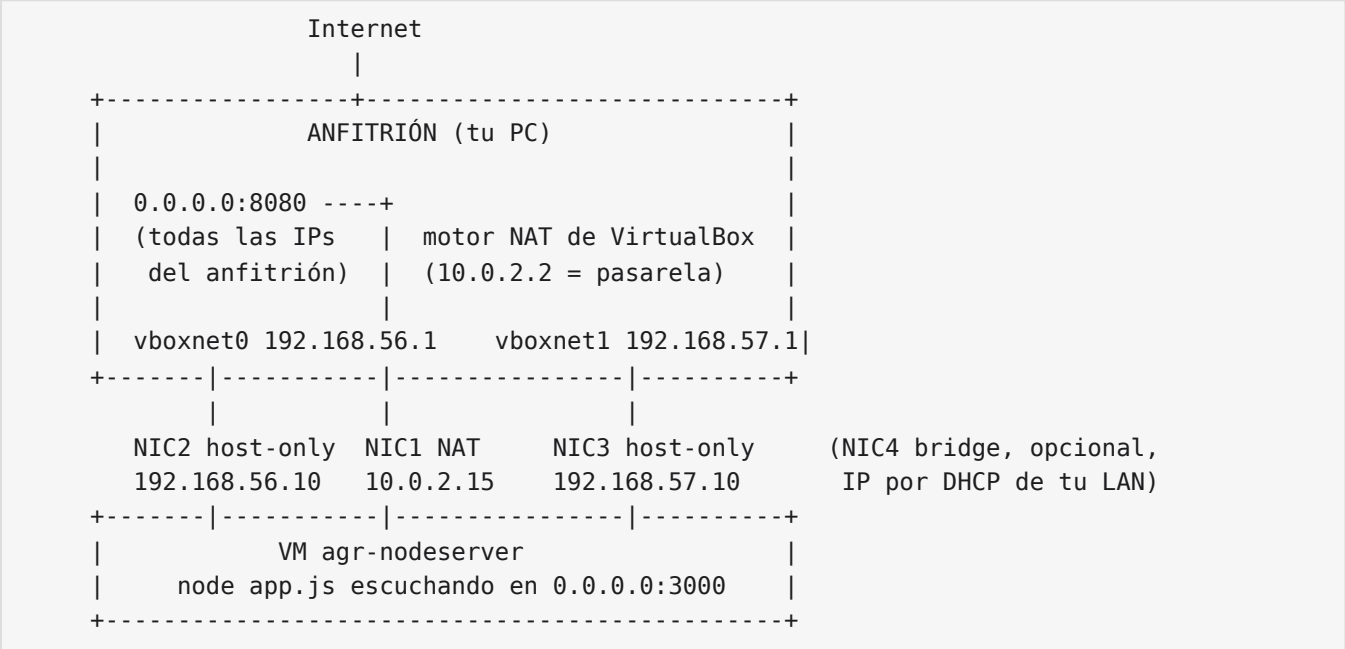
Este documento reúne en un solo sitio:

1. **Cómo están conectadas todas las redes** de las dos partes, con el recorrido de un paquete salto a salto.
2. **El cumplimiento del enunciado punto a punto**: qué está hecho, qué está a medias y qué falta.
3. **El proceso completo**: creación, integración, despliegue y testeo de la infraestructura, incluidos los fallos encontrados y su corrección.
4. **La explicación detallada de cada script y fichero** del repositorio.

1. Cómo están conectadas las redes

1.1. Parte 1: la VM de Vagrant en VirtualBox

La VM `agr-nodeserver` tiene hasta cuatro tarjetas de red. Cada una conecta la VM con un "mundo" distinto:



NIC	Tipo	Dirección en la VM	Para qué sirve
1	NAT	10.0.2.15 (la asigna VirtualBox)	Salida a Internet (apt, npm, git), SSH de Vagrant y **redirección de puertos**
2	host-only (`vboxnet0`)	192.168.56.10/24	Red privada anfitrión ↔ VM para el servicio web
3	host-only (`vboxnet1`)	192.168.57.10/24	Segunda red privada (gestión): demuestra una VM con varias interfaces
4	bridge (opcional)	DHCP de tu red física	La VM aparece como un equipo más de tu LAN (`BRIDGE_IF=<nic> vagrant up`)

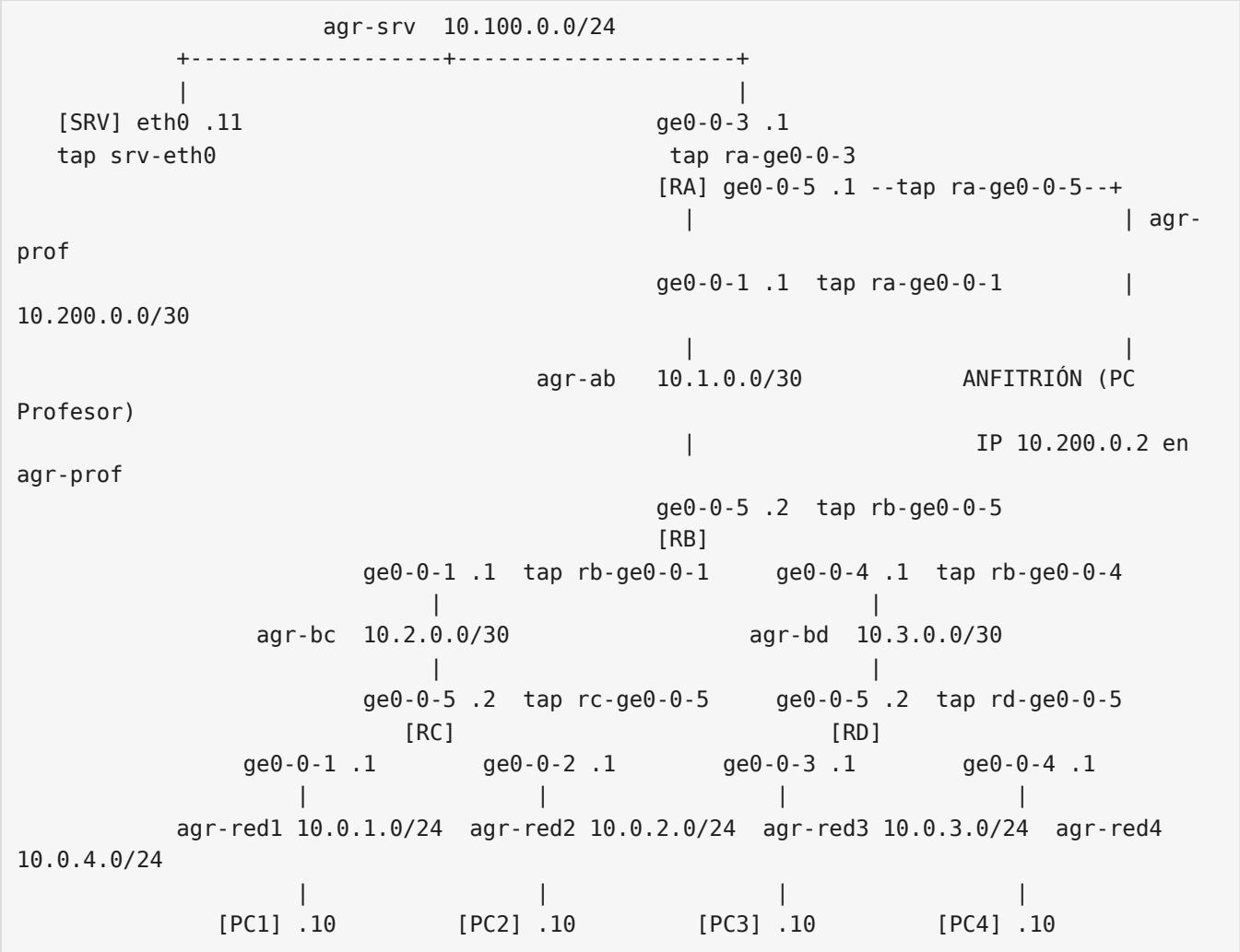
Cómo se llega al servidor web:

- **Desde la propia VM:** `http://localhost:3000`. Node escucha en todas las interfaces de la VM.
- **Desde el exterior por el puerto 8080:** VirtualBox abre el puerto 8080 en todas las IPs del anfitrión. Lo que llega ahí lo reenvía su motor NAT a `10.0.2.15:3000` dentro de la VM. Como no se fija `host_ip`, también entran otros equipos de tu red con `http://<IP-de-tu-PC>:8080`.
- **Por la red privada:** `http://192.168.56.10:3000` desde el anfitrión.
- **Cortafuegos:** dentro de la VM, `ufw` deniega todo lo entrante salvo 22/tcp (SSH) y 3000/tcp. Así se llega a la web y a SSH, y a nada más.

1.2. Parte 2: la réplica del laboratorio 3103 en KVM

Idea general

Cada **red** del plano es un **bridge Linux** en el anfitrión, que hace de switch virtual. Cada **interfaz** de una VM es un **tap**, un cable virtual que libvirt crea y conecta a su bridge. Los routers son VMs con varias interfaces y FRR, que encaminan entre redes según las tablas del plano.



(El mismo esquema, dibujado, está en `doc/entregables/src/img/topologia_kvm.svg` y en la memoria de la parte 2.)

Capa 2: bridges, taps y MAC

Bridge (switch)	Subred	Qué conecta (taps)
`agr-srv`	10.100.0.0/24	`ra-ge0-0-3` (RA), `srv-eth0` (servidor)
`agr-prof`	10.200.0.0/30	`ra-ge0-0-5` (RA) y la propia IP del anfitrión
`agr-ab`	10.1.0.0/30	`ra-ge0-0-1` (RA), `rb-ge0-0-5` (RB)
`agr-bc`	10.2.0.0/30	`rb-ge0-0-1` (RB), `rc-ge0-0-5` (RC)
`agr-bd`	10.3.0.0/30	`rb-ge0-0-4` (RB), `rd-ge0-0-5` (RD)
`agr-red1`	10.0.1.0/24	`rc-ge0-0-1` (RC), `pc1-eth0` (PC1)
`agr-red2`	10.0.2.0/24	`rc-ge0-0-2` (RC), `pc2-eth0` (PC2)
`agr-red3`	10.0.3.0/24	`rd-ge0-0-3` (RD), `pc3-eth0` (PC3)
`agr-red4`	10.0.4.0/24	`rd-ge0-0-4` (RD), `pc4-eth0` (PC4)

- **MAC fijas:** cada interfaz tiene la MAC `52:54:00:aa:<nº de máquina>:<nº de interfaz>`. El número de máquina sigue el orden de `topologia.json`: RA=01, RB=02, RC=03, RD=04, SRV=05, PC1..PC4=06..09. Por ejemplo, la segunda interfaz de RC es `52:54:00:aa:03:01`.
- **Por qué sirve:** dentro de la VM, `agr-net.sh` busca cada MAC y sabe qué interfaz es, sin depender del orden en que Linux detecte las tarjetas.
- **Nombres:** las interfaces se renombran como en el plano (`ge0-0-1`…). Linux no admite `/` en un nombre de interfaz, de ahí el `-`.
- **Dentro de una red** (por ejemplo, PC1 y RC en `agr-red1`), las tramas van de un tap al otro a través del bridge, igual que en un switch físico. ARP funciona normalmente.
- **Cortafuegos del anfitrión:** el script pone `bridge-nf-call-iptables=0`. Así el tráfico puenteado entre VMs no pasa por iptables del anfitrión (Docker o ufw podrían bloquearlo).

Capa 3: direcciones y puertas de enlace

Equipo	Interfaz	IP	Puerta de enlace / rutas
RA	ge0-0-3 / ge0-0-5 / ge0-0-1	10.100.0.1/24, 10.200.0.1/30, 10.1.0.1/30	por defecto → 10.1.0.2 (RB)
RB	ge0-0-5 / ge0-0-1 / ge0-0-4	10.1.0.2/30, 10.2.0.1/30, 10.3.0.1/30	10.0.1–2.0/24 → 10.2.0.2 (RC); 10.0.3–4.0/24 → 10.3.0.2 (RD); por defecto → 10.1.0.1 (RA)
RC	ge0-0-5 / ge0-0-1 / ge0-0-2	10.2.0.2/30, 10.0.1.1/24, 10.0.2.1/24	por defecto → 10.2.0.1 (RB)
RD	ge0-0-5 / ge0-0-3 / ge0-0-4	10.3.0.2/30, 10.0.3.1/24, 10.0.4.1/24	por defecto → 10.3.0.1 (RB)
SRV	eth0	10.100.0.11/24	por defecto → 10.100.0.1 (RA)
PC1–PC4	eth0	10.0.N.10/24	por defecto → 10.0.N.1 (RC o RD)
Anfitrión	agr-prof	10.200.0.2/30	10.100.0.0/24, 10.1–3.0.0/30 y 10.0.1–4.0/24 → 10.200.0.1 (RA)

Las rutas de los routers son exactamente las tablas del plano. Las directamente conectadas las pone el kernel al asignar la IP; las estáticas, FRR.

Recorrido de un paquete: PC1 pide la web al servidor

`curl http://10.100.0.11:3000/` en PC1 (10.0.1.10):

#	Dónde	Decisión	Sale por
1	PC1	10.100.0.11 no está en 10.0.1.0/24 → ruta por defecto vía 10.0.1.1. ARP: "¿quién tiene 10.0.1.1?" → RC responde con 52:54:00:aa:03:01	`pc1-eth0` → bridge `agr-red1` → `rc-ge0-0-1`
2	RC	Reenvío activado. 10.100.0.11 no es directa → ruta por defecto vía 10.2.0.1	`ge0-0-5` → `agr-bc` → RB
3	RB	No coincide con 10.0.x → ruta por defecto vía 10.1.0.1	`ge0-0-5` → `agr-ab` → RA
4	RA	10.100.0.0/24 está directamente conectada en ge0-0-3 → ARP por 10.100.0.11	`ge0-0-3` → `agr-srv` → SRV
5	SRV	Node (puerto 3000) responde a 10.0.1.10 → ruta por defecto vía 10.100.0.1	`eth0` → RA
6	RA	10.0.1.10 no es directa → ruta por defecto vía 10.1.0.2	→ RB
7	RB	10.0.1.0/24 → vía 10.2.0.2	→ RC
8	RC	10.0.1.0/24 directamente conectada en ge0-0-1	→ PC1

Es exactamente lo que mide el `traceroute` del plan de pruebas: `10.0.1.1 → 10.2.0.1 → 10.1.0.1 → 10.100.0.11`.

Los otros caminos:

- **Anfitrión → PC3:** el anfitrión envía a RA (su ruta hacia 10.0.3.0/24). RA usa su ruta por defecto hacia RB, RB usa 10.0.3.0/24 vía RD, y RD entrega a PC3 porque es su red directa. La vuelta: PC3 → RD → RB → RA, y RA tiene directamente conectada la red 10.200.0.0/30 del anfitrión.
- **PC1 → PC4** (entre redes inferiores): PC1 → RC (ruta por defecto) → RB (10.0.4.0/24 vía RD) → RD → PC4. Funciona sin configuración adicional porque así lo dictan las tablas del plano, y es lo que pide el enunciado ("conectividad global").
- **Un destino que no existe** (por ejemplo 8.8.8.8): RC → RB → RA → RB → RA… RA y RB se envían el paquete mutuamente por sus rutas por defecto hasta que se agota el TTL y vuelve un ICMP "time exceeded". El plano no muestra ninguna salida a Internet (presumiblemente estaría tras RA en el laboratorio real) y la réplica no la tiene. Las VMs no tienen Internet: el software se instala en la imagen base antes de desplegar.

- **Tu red física y tu Internet no se tocan:** el anfitrión sólo añade rutas hacia las subredes del laboratorio, ninguna ruta por defecto.

2. Cumplimiento del enunciado, punto por punto

Fuente: `doc/development/enunciado/Practica_2_enunciado.pdf`.

Leyenda:

- ✅ hecho y comprobado.
- ⚠️ hecho con matices, o sin verificar en tu PC.
- ❌ no hecho.

2.1. Parte 1: “Virtualización de servicios web con Vagrant” (págs. 1–2)

#	Requisito del enunciado	Estado	Dónde / evidencia
1.1	Script Vagrant (Ruby) que despliegue una VM Linux (distribución libre)	✅	`parte1-vagrant/Vagrantfile`, Ubuntu 22.04
1.2	Servidor web Node.js accesible desde la propia VM y desde el exterior	✅	Probado en tu PC: `curl localhost:8080` y `curl localhost:3000` dentro de la VM
1.3	El script instala Node, descarga de GitHub el servidor y lo ejecuta	✅	*Provisioners* `node` y `servidor`; `ssr-server.service`
1.4	Usar SSR-master-server (o uno propio)	✅	Se clona `gisai/SSR-master-server`
1.5	Redirección de puertos: exterior 8080, VM 3000	✅	`forwarded_port guest: 3000, host: 8080`
1.6	Asignar una dirección de red acorde a la conexión (pública o privada)	✅	Privada 192.168.56.10 (y 192.168.57.10); pública opcional por DHCP
1.7	Entregar el script Ruby	✅	`parte1-vagrant/Vagrantfile`
1.8	(Valorado) configuraciones adicionales y esquemas de red avanzados, p. ej. varias interfaces	✅	3–4 NICs, `ufw`, usuario sin privilegios, teclado `es`, información al iniciar sesión, tope de CPU, clon enlazado
1.9	Estudiar el consumo de memoria y CPU del servicio virtualizado	✅	`memoria_parte1` §4.1–4.2, con tus medidas (`doc/development/parte1/consumo_raw.txt`)
1.10	Discutir el consumo de una red que replique el laboratorio (dibujo de Moodle)	✅	`memoria_parte1` §4.4 (réplica completa 38 VMs y reducida 9)
1.11	Vigilar con `watch` cómo crecen las imágenes	⚠️	El crecimiento se midió con `ls`/`du` antes y después de la carga de 25 000 peticiones (el `.vmdk` no creció de forma visible, §4.3). No consta una captura de `watch` durante la

#	Requisito del enunciado	Estado	Dónde / evidencia
			carga: si la quieres en la memoria, basta con repetirlo con `watch` y pegar la salida
1.12	(Valorado) comparar con servidores comerciales actuales	✅	`memoria_parte1` §4.5
1.13	(Opcional) documento con pruebas y configuraciones	✅	`memoria_parte1.pdf`, `manual_usuario.pdf`, `creacion_maquinas_virtuales.pdf`
—	Teclado español e impresión de la configuración al iniciar sesión (petición tuya)	✅	*Provisioners* `sistema` y `login-info`

2.2. Parte 2: “Virtualización de infraestructura de red con KVM” (págs.

3–4)

#	Requisito del enunciado	Estado	Dónde / evidencia
2.1	Script que despliegue la red del dibujo (lab. 3103) con QEMU-KVM	✅	`parte2-kvm/agr_kvm.py crear`
2.2	Lenguaje libre (se aconseja Python y Bash)	✅	Python 3 (orquestación) + sh (dentro de las VMs)
2.3	Generar los XML necesarios para todas las VMs	✅	`generar_xml()`; ejemplos en `doc/development/parte2/xml_generados/`, validados con `virt-xml-validate`
2.4	Las 4 subredes pueden ser un único PC (o el anfitrión)	✅	Un PC por red (PC1–PC4)
2.5	Usar una librería XML de Python (se aconseja `lxml.etree`)	✅	`lxml.etree` en `generar_xml()`
2.6	Desplegar todas las máquinas	✅	9 VMs: RA, RB, RC, RD, SRV, PC1–PC4
2.7	Instalar la *Software Routing Suite* en los routers	✅	FRRouting (FRR 7.2 en Ubuntu 20.04), sucesor de Quagga, activo sólo en RA–RD. Si el profesor exige Quagga literalmente, se puede cambiar (Ubuntu 20.04 aún lo incluye)
2.8	Rutas estáticas para conectividad global	✅	`frr.conf` por router = tablas del plano; 50/50 pruebas
2.9	Servicio web en el servidor (reutilizando la parte 1)	✅	SSR-master-server en SRV (10.100.0.11:3000)
2.10	Cualquier máquina de la parte inferior accede al servicio web	✅	Pruebas PC1–PC4 → HTTP 200 y escritura en `/record`
2.11	El anfitrión accede al servidor	✅	Prueba anfitrión → HTTP 200
2.12	Entregar el script	✅	`parte2-kvm/`
2.13	Memoria con el plan de numeración y un dibujo	✅	`memoria_parte2` §3 + `topologia_kvm.svg`
2.14	Memoria con las tablas de	✅	`memoria_parte2` §4 (más `show ip route` real en §5.1)

#	Requisito del enunciado	Estado	Dónde / evidencia
	enrutamiento de los routers		
2.15	Plan de pruebas que confirme la conectividad global	✅	`agr_kvm.py probar` (50 pruebas); `memoria_parte2` §5; `doc/development/parte2/pruebas.txt`
2.16	(Valorado) configuraciones adicionales en las VMs	✅	Nombres de interfaz del plano, información al iniciar sesión, teclado `es`, SSH con clave, agente QEMU, discos diferenciales
2.17	(Valorado) configuraciones adicionales en la *Software Routing Suite*	⚠️	Hay pocas: descripción de interfaces, `ip forwarding`, *logging*, plazos de `watchfrr`. Se podría añadir más (ver §2.5)

2.3. “Propuesta de solución” y “Recomendaciones” (pág. 4)

#	Indicación	Estado	Comentario
3.1	Usar como base la **imagen de disco** de Moodle	⚠️	Es la imagen por defecto del script (`parte2-kvm/imagen/agr-vm-base.qcow2`). Probado con una imagen Ubuntu 20.04 equivalente; **falta probarlo con tu imagen exacta**
3.2	Usar como base el **XML** de Moodle	✅	`plantilla_vm.xml` es el XML de Moodle; el script parte de él
3.3	root / agr	✅	Se mantiene
3.4	Se puede cambiar hostname y contraseña	✅	Hostname cambiado en cada VM (`ra`, `rb`…)
3.5	Aconsejado: conectar el **anfitrión a un bridge de las subredes inferiores** (RED 1–4)	❌	El anfitrión está en 10.200.0.0/30 como “PC Profesor”, siguiendo el plano. **Pendiente de que decidas** si lo cambio
3.6	Mensajes fijos con `print`; trazas de depuración con `logging`	✅	`info()` usa `print`; `log.debug/info/warning` con `-v`
3.7	Gestionar las VMs con `virsh define/start/shutdown/destroy`	✅	`crear` (define + start), `parar` (shutdown), `destruir` (destroy + undefine)
3.8	Consolas con `xterm -e "virsh console …" &` en segundo plano	✅	`consola` lanza `xterm` en segundo plano (`Popen(start_new_session=True)`, equivalente a `&`)
3.9	`virt-cat` / `virt-ls` para inspeccionar sin entrar	⚠️	No se usan; se usa `virt-customize` (de la misma familia, libguestfs) para escribir en los discos y el agente QEMU para inspeccionar las VMs en marcha

2.4. Lo que falta o queda por confirmar

1. **Probar en tu PC con la imagen de Moodle real** (punto 3.1): copiarla a `parte2-kvm/imagen/agr-vm-base.qcow2` y ejecutar `instalar`, `crear` y `probar`.
2. **Medir la RAM con esa imagen**: `sudo ./agr_kvm.py consumo --informe ../doc/development/parte2/consumo.txt`. La tabla de memoria de la memoria es de la versión anterior (Ubuntu 24.04) y así se indica en el texto.
3. **Decidir dónde va el anfitrión** (punto 3.5): como PC Profesor (ahora) o en una de las redes inferiores, como aconseja el enunciado.
4. **(Opcional) Subir la imagen al Release** `imagen-moodle`, para que el repositorio la descargue solo en cualquier PC.
5. **(Opcional) Más configuración de FRR** (punto 2.17). Ver §2.5.
6. **Limpieza:** borrar `asdasd.txt` (fichero vacío) y revisar el autor de los documentos.

2.5. Propuestas para reforzar el punto 2.17 (FRR)

Las rutas exigidas ya están. Para sumar en "configuraciones adicionales" de la *Software Routing Suite* se podría añadir, sin cambiar el plano:

- **Ruta *blackhole* en RA para destinos desconocidos.** Evita el bucle RA↔RB.
- **Contraseña y banner en `vtysh`/vty.**
- **Ruta de respaldo con distancia administrativa mayor**, para demostrar *floating static routes*.
- **OSPF como alternativa** a las rutas estáticas, en una variante documentada.

Dime cuáles quieres y las incorporo.

3. Proceso completo: creación, integración, despliegue y testeo

3.1. Parte 1 (Vagrant)

Creación.

- Se eligió `bento/ubuntu-22.04`: una *box* mínima y mantenida.
- El `Vagrantfile` declara red, hardware y cuatro *provisioners* Bash.

Integración del servidor.

- Se analizó el repositorio SSR-master-server. `app.js` hace `app.listen(3000)` y `bin/www` (lo que lanza `npm start`) vuelve a escuchar en el 3000, así que `npm start` muere con `EADDRINUSE`. Se comprobó ejecutándolo.
- Por eso el servicio systemd lanza `node app.js` directamente.

Despliegue en tu PC. Hubo que resolver, en este orden:

1. VirtualBox no cargaba su módulo: tu kernel 7.0 era demasiado nuevo para VirtualBox 7.1. Se actualizó a la 7.2.
2. Secure Boot rechazaba el módulo sin firmar. Se desactivó.
3. KVM y VirtualBox compiten por la virtualización. Ahora lo gestionan los propios scripts.

Testeo.

- `curl` desde la VM, desde el anfitrión por el puerto 8080 y por la red privada.
- Escritura en `/record`.
- Carga de 25 000 peticiones.
- Medidas de consumo (`ps`, `VBoxManage metrics`, `free`, `du`) que dan pie al estudio de la memoria.

3.2. Parte 2 (KVM)

Creación (diseño).

1. Se transcribió el plano a `topologia.json`: 9 redes, 9 máquinas, IPs y rutas tal cual. El script valida que cada IP pertenezca a su red.
2. Decisiones:
    - un PC por red inferior;
    - el anfitrión como PC Profesor;
    - interfaces con el nombre del plano;
    - MAC deterministas;
    - una sola imagen base con todo el software y un disco diferencial por VM.
3. Se eligió FRR como *Software Routing Suite* (Quagga está abandonado; FRR es su continuación) y el agente QEMU para poder probar desde el anfitrión sin depender de la red ni de SSH.

Integración (qué se mete en las VMs y cómo).

1. **Imagen base:** una copia de la imagen de Moodle ampliada a 6 GB. Con `virt-customize` se instala dentro, sin arrancarla, todo lo común: FRR, Node.js, SSR-master-server, agente QEMU, el servicio de red `agr-net` y el aviso al iniciar sesión.
2. **Disco por VM:** un qcow2 diferencial sobre la base. Con `virt-customize` se escribe lo propio de cada máquina: hostname, `node.conf`, `/etc/hosts`, `frr.conf` y servicios del rol, y la clave SSH.
3. **XML por VM:** se genera desde la plantilla de Moodle con `lxml` (nombre, RAM, CPU, disco, una interfaz por red con su MAC y su tap, canal del agente y teclado `es`).

Despliegue (`sudo ./agr_kvm.py crear`).

1. Comprueba que eres root y que están las herramientas.
2. Prepara el hipervisor: descarga VirtualBox, carga `kvm_amd`/`kvm_intel` y arranca `libvirtd`.
3. Construye la imagen base, sólo si no existe.
4. Crea los 9 bridges, pone 10.200.0.2 al anfitrión y sus rutas hacia el laboratorio.
5. Para cada VM:
    1. Si ya existía, la destruye.
    2. Crea su disco diferencial y lo configura.
    3. Genera su XML.
    4. Ejecuta `virsh define` y `virsh start`.
6. Espera a que cada VM esté **lista**: el agente responde, `agr-net` ha terminado, tiene todas sus rutas y, en el servidor, la web responde.
7. Muestra el estado.

Testeo (`sudo ./agr_kvm.py probar`). 50 pruebas automáticas:

Bloque	Número de pruebas
Anfitrión → 17 interfaces (ping) + web	18
PC1–PC4 → servidor (ping, HTTP, escritura, traceroute)	16
Entre las 4 redes inferiores	12
Tablas de rutas de los 4 routers	4

Resultado: **50/50**, con el informe en `doc/development/parte2/pruebas.txt`.

Fallos encontrados durante el testeo y su corrección:

Síntoma	Causa	Corrección
PC1 y PC2 sin conectividad tras arrancar todo a la vez	`watchfrr` reinició `staticd` por tardar en responder, y un demonio reiniciado por separado no recarga la configuración: RC perdió su ruta por defecto	Plazos de `watchfrr` ampliados y `vtysh -b` (reaplicar la configuración) al arrancar FRR. Probado reiniciando los 4 routers a la vez
La prueba de `traceroute` daba OK aunque fallara	La cabecera de `traceroute` ya contiene la IP de destino	Se comprueba el último salto
Pruebas lanzadas antes de que los routers tuvieran rutas	La espera sólo miraba el servicio de red	La espera exige todas las rutas instaladas
La primera petición web del anfitrión fallaba	Arranque en frío de Node (2.8 s)	La espera exige que la web responda; los HTTP admiten 15 s
`virt-customize` sin red	Faltaba el cliente DHCP del entorno de libguestfs	`isc-dhcp-client` en las dependencias (`instalar`)
VMs sin acceso a su disco	Directorio de discos no accesible por `libvirt-qemu`	Se usa `/var/lib/libvirt/images/agr` y el script avisa si no

4. Explicación detallada de cada script y fichero

4.1. `parte1-vagrant/Vagrantfile` (Ruby)

Cabecera y parámetros. Constantes Ruby con todos los valores de la práctica:

- nombre y *box*;
- RAM y CPU (se pueden cambiar con `VM_MEMORY`/`VM_CPUS`);
- las dos IPs privadas;
- los puertos 3000 y 8080;
- la URL del repositorio, la carpeta de instalación (`/opt/ssr-master-server`), el usuario de servicio (`nodeapp`) y la versión de Node (22).

`ENV[...]` permite cambiarlos sin editar el fichero.

`SCRIPT_SISTEMA` (*heredoc* Bash). Empieza con `set -euo pipefail`, que detiene el script ante cualquier error, y luego:

1. **Teclado:** escribe `/etc/default/keyboard` con `XKBLAYOUT="es"`, instala `keyboard-configuration`, `console-setup`, `kbd` y `locales`, y lo aplica (`dpkg-reconfigure`, `setupcon`, `localectl set-keymap es`, `localectl set-x11-keymap es`, `loadkeys es`).
2. **Idioma y hora:** genera `es_ES.UTF-8`, lo pone por defecto y fija `Europe/Madrid`.
3. **Paquetes:** instala `git`, `curl`, `ufw`, `htop`, `sysstat` y `net-tools`.
4. **Cortafuegos `ufw`:** lo reinicia, deniega la entrada por defecto, permite 22/tcp y 3000/tcp y lo activa. `#{PUERTO_GUEST}` es interpolación de Ruby: se sustituye antes de enviar el script a la VM.

`SCRIPT_NODE`. Si `node -v` no empieza por `v22.`, añade el repositorio de NodeSource y instala `nodejs`. Así, al repetir `vagrant provision` no reinstala nada.

`SCRIPT_SERVIDOR`:

1. Crea el usuario de sistema `nodeapp`, sin shell.
2. Clona el repositorio (o hace `git pull` si ya existe).
3. Crea `public/logs`, le da la propiedad a `nodeapp` y ejecuta `npm install --omit=dev` como ese usuario.
4. Escribe `/etc/systemd/system/ssr-server.service`. Esta unidad lanza `node app.js` como `nodeapp` en el directorio de la aplicación, con `Restart=on-failure`.
5. Ejecuta `daemon-reload`, `enable` y `restart`.
6. Espera hasta 20 s a que `curl localhost:3000` responda.

`SCRIPT_LOGIN_INFO`. Quita el permiso de ejecución a los scripts del MOTD de Ubuntu e instala `/etc/profile.d/99-servidor-info.sh`. Ese fichero se ejecuta en cada inicio de sesión interactivo (`case $- in *i*`) e imprime:

- **Sistema:** hostname, SO, kernel, CPUs, memoria, disco, carga, teclado y zona horaria.
- **Red:** IPs, pasarela y DNS.
- **Servidor web:** estado del servicio, versiones de Node/npm, URLs desde la VM, la red privada y el anfitrión, y los puertos en escucha.
- **Cortafuegos:** las reglas de `ufw`.

`Vagrant.configure("2")`:

- `ENV["VAGRANT_DEFAULT_PROVIDER"] ||= "virtualbox"`: fuerza VirtualBox, de modo que los errores sean concretos.
- **Trigger** `config.trigger.before [:up, :reload, :resume]`: antes de arrancar, ejecuta en el anfitrión un `bash -c` que descarga `kvm_amd`, `kvm_intel` y `kvm` si están cargados y carga `vboxdrv` si no lo está. Si KVM está en uso, avisa y aborta. Por eso puede pedir la contraseña de `sudo`.
- `forwarded_port guest: 3000, host: 8080`: la redirección de puertos (con `id` para identificarla).
- Dos `private_network` (host-only) y un `public_network` opcional.
- Bloque `provider "virtualbox"`:
  - `name`, `gui`, `memory` y `cpus`;
  - `linked_clone = true`: disco diferencial sobre la *box*;
  - `--cpuexecutioncap 80`: la VM no usa más del 80 % de un núcleo;
  - `--natdnshostresolver1 on`: la VM resuelve DNS con el del anfitrión;
  - una descripción visible en VirtualBox.
- Los cuatro `provision ... type: "shell", inline:` en orden.
- `post_up_message` con las URLs.

4.2. `parte2-kvm/agr_kvm.py` (Python 3)

Constantes (líneas 45–68)

Constante	Valor y uso
`AQUI`, `GUEST`	Carpeta del script y `guest/` (ficheros que se suben a las VMs)
`PREFIJO`	`agr-`: los dominios libvirt se llaman `agr-ra`, `agr-pc1`…
`IMAGEN_MOODLE`	`parte2-kvm/imagen/agr-vm-base.qcow2`: imagen de partida
`URL_IMAGEN_MOODLE`	Descarga desde el Release `imagen-moodle` si no hay copia local
`URL_IMAGEN_UBUNTU`	Alternativa explícita (`--imagen-base ubuntu`)
`DEPENDENCIAS`	Paquetes que instala la orden `instalar`
`CONTRASENA_ROOT`	`agr`
`DIR_TRABAJO`	`/var/lib/libvirt/images/agr`: imagen base, discos y XML
`TAMANO_DISCO`	`6G`: tamaño al que se amplía la imagen base
`AMPLIAR_RAIZ`	Orden de shell que localiza la partición raíz (`findmnt`, `lsblk`, `/sys/class/block/*/partition`) y la amplía con `growpart` + `resize2fs`, sea cual sea la disposición de la imagen

Utilidades

- **`sh(cmd, check, capture, env)`:** ejecuta un comando con `subprocess.run`.
  - Lo registra en el log de depuración (`-v`).
  - Si `check=True` y falla, lanza `RuntimeError` con el comando y su error. `main()` lo captura y termina con un mensaje limpio.
- **`virsh(*args)`:** atajo de `sh(["virsh", "-c", "qemu:///system", ...])`. Siempre usa la conexión del sistema.
- **`info(msg)`:** mensajes fijos para el usuario, con `print` y el prefijo `>>>` en color (la recomendación del enunciado).

Comprobación y preparación del anfitrión

- **`comprobar_root()`:** sale si no se ejecuta como root.
- **`comprobar_entorno()`:** root, presencia de `virsh`, `qemu-img`, `virt-customize` e `ip` (si faltan, remite a `instalar`). Después llama a `preparar_hipervisor()` y fija `LIBGUESTFS_BACKEND=direct`, para que libguestfs lance su QEMU directamente sin pasar por libvirt.
- **`preparar_hipervisor()`:**
  1. Si `vboxdrv` está cargado, intenta descargar `vboxnetadp`, `vboxnetflt` y `vboxdrv`. Si no puede (hay VMs de VirtualBox encendidas), avisa.
  2. Si no existe `/dev/kvm`, mira en `/proc/cpuinfo` si la CPU es AMD (`svm`) o Intel (`vmx`) y carga `kvm_amd` o `kvm_intel`. Si aun así no hay `/dev/kvm`, avisa de que se usará emulación software (virtualización desactivada en la BIOS).
  3. `systemctl start libvirtd`.

- **`cmd_instalar()`:** `apt-get update` + `apt-get install -y` de `DEPENDENCIAS` y `systemctl enable --now libvirtd`.

Topología

- **`cargar_topologia(ruta)`:** lee `topologia.json` y valida con `ipaddress` que la IP de cada interfaz pertenece a la subred de su red. Si no, lanza un error antes de tocar nada.
- **`mac(i, j)`:** devuelve `52:54:00:aa:{i:02x}:{j:02x}`. `52:54:00` es el prefijo de QEMU.
- **`dominio(nombre)`:** `agr-` + nombre.

Red del anfitrión

- **`crear_bridges(topo)`:**
  1. Para cada red: si el bridge no existe, `ip link add name <br> type bridge`; después `ip link set <br> up`. En la red marcada `anfitrion` (`agr-prof`) asigna `10.200.0.2/30` con `ip addr replace`, que es idempotente.
  2. Pone a 0 `bridge-nf-call-iptables`, `-ip6tables` y `-arptables`, si existen.
  3. Calcula la pasarela: la IP de la VM conectada a la red del anfitrión, es decir, RA 10.200.0.1.
  4. Para cada otra red: `ip route replace <cidr> via 10.200.0.1 dev agr-prof`. Si ya había una ruta distinta a esa subred, avisa de que la reemplaza.
- **`borrar_bridges(topo)`:** `ip route del` e `ip link del` de cada red.

Imágenes

- **`descargar(url, destino)`:** `curl -fL` a un fichero `.part`, que sólo se renombra si la descarga termina bien. Nunca queda una imagen a medias.
- **`imagen_origen(args)`:** decide la imagen de partida, por este orden:
  1. `--imagen-base ubuntu`: descarga la de Ubuntu 24.04.
  2. `--imagen-base RUTA`: esa imagen.
  3. `imagen/agr-vm-base.qcow2`: la de Moodle local.
  4. La copia ya descargada en `DIR_TRABAJO`, o el Release `imagen-moodle`.
  5. Si nada de lo anterior existe, error explicando dónde poner la imagen.
- **`imagen_base(args)`:** si `agr-base.qcow2` existe (y no se pide `--reconstruir-base`), la reutiliza. Si no:
  1. `qemu-img convert` a un `.tmp`: copia independiente; el original nunca se toca.
  2. `qemu-img resize` a 6 GB.
  3. Una sola llamada a `virt-customize --network` que:
     - amplía la raíz (`AMPLIAR_RAIZ`);
     - pone la contraseña de root y la zona horaria;
     - sube `agr-net.sh` (con permisos 0755), `agr-net.service`, `ssr-server.service`, `agr-info.sh` y `build-base.sh`;
     - ejecuta `build-base.sh` (con `AGR_PROXY` si se pasó `--proxy`);
     - borra `build-base.sh`.
  4. Si todo fue bien, renombra el `.tmp` a `agr-base.qcow2`. Una construcción interrumpida nunca deja una base rota.

Ficheros de configuración que se generan

- **`frr_conf(nombre, m, topo):`** genera el `frr.conf` del router:
  - `frr defaults traditional`;
  - `hostname RA`;
  - `log syslog informational`;
  - `service integrated-vtysh-config` (un único fichero para todos los demonios);
  - `ip forwarding`;
  - un bloque `interface ge0-0-x` con `description` por interfaz;
  - una línea `ip route <destino> <next-hop>` por ruta de `topologia.json`;
  - `line vty`.
- **`node_conf(nombre, idx, m):`** genera `/etc/agr/node.conf`: `role <rol>`, una línea `iface <mac> <nombre> <ip/prefijo>` por interfaz y una línea `route <destino> <next-hop>` por ruta.
- **`hosts_file(topo):`** genera `/etc/hosts` con `localhost`, `profesor` y, por cada interfaz, `<ip> <máquina>-<interfaz>`. La primera interfaz lleva además el nombre corto: `ping srv`, `ping rb-ge0-0-4`…
- **`ssh_pubkey():`** busca la clave pública (`id_ed25519`, `id_rsa` o `id_ecdsa`) del usuario que ejecutó `sudo` (`SUDO_USER`), para inyectarla en root.

Disco de cada VM

`crear_disco(nombre, idx, m, topo, base, args):`

1. Borra el disco anterior, si lo hay.
2. `qemu-img create -f qcow2 -F qcow2 -b agr-base.qcow2 <vm>.qcow2`: disco diferencial.
3. En un directorio temporal genera `node.conf` y `hosts` (y `frr.conf` si es router).
4. Lanza `virt-customize -q -a <vm>.qcow2`, sin red, con:
   - `--hostname`;
   - `--mkdir /etc/agr`;
   - `--upload` de `node.conf` y `hosts`;
   - en routers: `--upload` de `frr.conf` y `chown frr:frr … && systemctl enable frr.service`;
   - en el servidor: `systemctl enable ssr-server.service`;
   - si hay clave, `--ssh-inject root:file:<clave>`.

XML

`generar_xml(nombre, idx, m, topo, disco, plantilla):`

1. Importa `lxml.etree` aquí (y no al principio), para que `instalar` funcione aunque `lxml` no esté instalado.
2. Parsea la plantilla. Si no hay `/dev/kvm`, cambia `type` a `qemu`.
3. Pone `<name>`, añade `<description>` tras el nombre y fija `<memory>` y `<currentMemory>` (MB × 1024 = KiB) y `<vcpu>`.
4. Si el `<emulator>` de la plantilla (`/usr/bin/kvm`) no existe, usa `qemu-system-x86_64`.
5. Pone la ruta del disco en `disk/source`.
6. Quita la `<interface>` de la plantilla y, en su misma posición, inserta una por cada interfaz de la VM: `type="bridge"`, `<mac>`, `<source bridge>`, `<target dev="ra-ge0-0-3">` (recortado a 15 caracteres, el máximo de Linux) y `<model virtio>`.
7. Añade `<channel type="unix">` con el destino `org.qemu.guest_agent.0` (el agente QEMU) y `keymap="es"` en `<graphics>`.

8. Devuelve el XML con sangría.

Órdenes

- **`maquinas_sel()`:** las máquinas indicadas en la orden, o todas. Error si el nombre no existe.
- **`cmd_xml()`:** genera los XML, `frr.conf` y `node.conf` en una carpeta, sin desplegar nada y sin root. Sirve para revisar.
- **`cmd_crear()`:**
  1. Prepara el entorno y avisa si `--dir` no es accesible para `libvirt-qemu`.
  2. `imagen_base` y `crear_bridges`.
  3. Por cada VM seleccionada: `destroy` + `undefine` si existía, `crear_disco`, `generar_xml`, guarda el XML en `DIR_TRABAJO`, `virsh define` y `virsh start`.
  4. `esperar_agentes` y `cmd_estado`.
- **`cmd_arrancar()`:** prepara el hipervisor, recrea bridges y rutas (no sobreviven a un reinicio del PC) y ejecuta `virsh start`.
- **`liberar_kvm()`:** si no queda ninguna VM encendida, descarga `kvm_amd`, `kvm_intel` y `kvm`, para que VirtualBox pueda usar la virtualización.
- **`cmd_parar()`:** `virsh shutdown` (apagado ordenado por ACPI), espera hasta 2 minutos a que se apaguen y llama a `liberar_kvm`.
- **`cmd_destruir()`:**
  1. `virsh destroy` + `undefine` y borra disco y XML.
  2. Sin máquinas concretas, borra también bridges y rutas.
  3. Con `--todo`, borra `DIR_TRABAJO` entero (imagen base incluida; nunca la de `imagen/`).
  4. `liberar_kvm`.
- **`cmd_estado()`:** `virsh domstate` de cada VM, con rol, RAM e IPs.
- **`rss_qemu()`:** recorre `/proc/<pid>/cmdline` buscando el argumento `guest=agr-…` de cada proceso QEMU y lee su `VmRSS` de `/proc/<pid>/status`. Es la RAM real que ocupa cada VM en el anfitrión.
- **`cmd_consumo()`:** por VM muestra la RAM asignada, el RSS, el disco real del qcow2 (`st_blocks × 512`, no el tamaño virtual) y el tiempo de CPU (`virsh domstats --cpu-total`). Después, los totales, la imagen base y el porcentaje de la RAM del anfitrión. Opcionalmente lo guarda con `--informe`.
- **`cmd_consola()`:** lanza `xterm -e "virsh console agr-x"` en segundo plano (sesión propia) por cada VM.

Agente QEMU

- **`qga(dom, comando, argumentos)`:** envía un JSON al agente QEMU de la VM con `virsh qemu-agent-command` y devuelve el campo `return`.
- **`guest_exec(nombre, orden, timeout)`:**
  1. `guest-exec` con `/bin/sh -c <orden>` y `capture-output`: el agente devuelve un `pid`.
  2. Consulta `guest-exec-status` cada 0.5 s hasta que `exited` es verdadero.
  3. Decodifica de base64 la salida (`out-data`/`err-data`) y devuelve el código de salida y la salida. Así se ejecutan órdenes dentro de las VMs sin red de gestión ni SSH.
- **`esperar_agentes(topo, nombres, timeout)`:** hasta que pasa el plazo, comprueba cada VM pendiente. Una VM está lista cuando:
  - responde `guest-ping`;
  - `agr-net.service` está activo;

- tiene al menos tantas rutas `via` como indica la topología;
- si es el servidor, la web responde en `localhost:3000`.

Plan de pruebas

`cmd_probar()`:

1. Espera a que todas las VMs estén listas.
2. **Bloque 1:** desde el anfitrión, `ping -c 2` a las 17 IPs de las VMs y `curl` a la web (espera código 200).
3. **Bloque 2:** dentro de cada PC, con `guest_exec`:
   - `ping` al servidor;
   - `curl` (código 200);
   - `curl` a `/record?...` (la respuesta debe contener `Saving`);
   - `traceroute -n` (el último salto debe ser 10.100.0.11).
4. **Bloque 3:** `ping` de cada PC a los otros tres.
5. **Bloque 4:** `vtysh -c 'show ip route'` en cada router (o `ip route` si no hay FRR).
6. Cada resultado pasa por `anotar()`, que imprime OK o FALLO y lo acumula para el informe, con la salida de `traceroute` y de las tablas. Al final imprime `Resultado: x/y` y, con `--informe`, lo guarda en un fichero. El código de salida es 0 sólo si todo pasa.

`main()`

1. `argparse` con las opciones globales: `-v` (trazas de depuración), `--topologia`, `--plantilla` y `--dir`.
2. Una suborden por acción, cada una con sus opciones:
   - `crear`: `--imagen-base`, `--reconstruir-base`, `--proxy`, `--espera`;
   - `probar`: `--informe`, `--detalle`, `--espera`;
   - `consumo`: `--informe`;
   - `xml`: `--salida`.
3. Configura `logging` (nivel `DEBUG` con `-v`, `INFO` sin él) y carga y valida la topología.
4. Ejecuta la función de la suborden. Si salta un `RuntimeError`, lo muestra como error y termina con código 1.

4.3. `parte2-kvm/topologia.json`

- `redes`: para cada red, `bridge` (nombre del bridge Linux, como mucho 15 caracteres), `cidr`, `descripcion` y, sólo en la del anfitrión, `anfitrion` (la IP que toma tu PC).
- `maquinas`: para cada VM:
  - `rol` (`router`, `server` o `pc`), `memoria_mb` y `vcpus`;
  - `interfaces`: lista de `{nombre, red, ip}`, en el orden en que se numeran sus MAC;
  - `rutas`: lista de `[destino, next-hop]`.
- `servicio_web`: máquina, IP y puerto del servidor, que usan la espera y las pruebas.

Cambiar la red del laboratorio consiste en editar este fichero. Sólo hay dos IPs escritas fuera de él: la del anfitrión en el `/etc/hosts` que genera el script, y la del servidor en el texto de ayuda de `agr-info.sh`.

4.4. `parte2-kvm/plantilla_vm.xml` (el XML de Moodle)

Elemento	Significado
`<domain type="kvm">`	VM acelerada por KVM
`<name>`	Nombre del dominio (lo sustituye el script)
`<memory>`, `<currentMemory>`	RAM máxima y actual, en KiB
`<vcpu>`	Número de CPUs virtuales
`<os><type arch="x86_64">hvm</type><boot dev="hd"/>`	Virtualización completa; arranca del disco
`<features><acpi/><apic/><pae/>`	ACPI (necesario para el apagado ordenado con `virsh shutdown`), APIC y PAE
`<clock offset="utc"/>`	Reloj del invitado en UTC
`<on_poweroff>destroy`, `<on_reboot>restart`, `<on_crash>restart`	Qué hacer en cada evento
`<emulator>/usr/bin/kvm`	Binario de QEMU
`<disk type="file" device="disk">` + `<driver type="qcow2"/>` + `<target dev="vda" bus="virtio"/>`	Disco qcow2 como `vda` con el controlador paravirtualizado virtio
`<interface type="bridge">` + `<source bridge>` + `<model type="virtio">`	Tarjeta de red conectada a un bridge, virtio (el script pone una por red)
`<serial type="pty">` + `<console type="pty">`	Consola serie, la que usa `virsh console`
`<input type="mouse" bus="ps2"/>`	Ratón
`<graphics type="vnc" autoport="yes">`	Pantalla por VNC (el script añade `keymap="es"`)

4.5. `parte2-kvm/guest/build-base.sh` (se ejecuta una vez dentro de la imagen base)

1. `set -eu` (se para ante errores o variables no definidas) y `DEBIAN_FRONTEND=noninteractive` (apt no pregunta nada).
2. **Proxy (opcional):** si `AGR_PROXY` está definido:
   - exporta `http_proxy` y `https_proxy`;
   - fija `NODE_EXTRA_CA_CERTS`, para que npm confíe en las CA del sistema tras un proxy con inspección TLS;
   - crea `/etc/apt/apt.conf.d/99agr-proxy`.
3. **Paquetes:** `apt-get update` e instalación sin recomendados de:
   - `frr` y `frr-pythontools` (la *Software Routing Suite*);
   - `qemu-guest-agent`;
   - herramientas de red: `iproute2`, `iputils-ping`, `traceroute`, `tcpdump`, `curl`;
   - otras utilidades: `git`, `xz-utils`, `procps`, `less`, `nano` y las de teclado.
4. **FRR:**
   - activa `staticd` en `/etc/frr/daemons`;
   - pone `watchfrr_options="-t 60 -T 90"`: más paciencia antes de dar por muerto un demonio y al reiniciarlo;
   - crea la unidad complementaria `frr.service.d/agr.conf`, cuyo `ExecStartPost` espera a que `vtysh` responda y ejecuta `vtysh -b` (aplica `frr.conf` a todos los demonios);

    - deshabilita `frr` (sólo se habilitará en los routers).
5. **Node.js:**
    - lee la versión de glibc;
    - si es 2.28 o mayor (Ubuntu 20.04 tiene 2.31), descarga Node 22.12.0 oficial; si no, Node 16;
    - lo descomprime en `/opt`, enlaza `/opt/node` y `/usr/local/bin/node` y muestra la versión.
6. **Servidor:**
    - crea `nodeapp`;
    - clona SSR-master-server en `/opt/ssr-master-server`;
    - crea `public/logs` y ejecuta `npm install --omit=dev` con el Node de `/opt/node`;
    - da la propiedad a `nodeapp` y deshabilita `ssr-server`.
7. **Red en manos de `agr-net`:**
    - desactiva cloud-init (`/etc/cloud/cloud-init.disabled`) y borra los YAML de netplan;
    - deja `/etc/network/interfaces` sólo con `lo`: la imagen de Moodle traía `eth0` por DHCP;
    - deshabilita systemd-networkd y NetworkManager y enmascara sus `wait-online`, para que el arranque no espere a una red que no van a configurar;
    - habilita `agr-net.service`. Resultado: nadie intenta DHCP en las redes del laboratorio.
8. **Teclado:** escribe `/etc/default/keyboard` con `es`.
9. **Limpieza:**
    - quita el proxy de apt y vacía cachés y listas de apt;
    - vacía `/etc/machine-id`, para que cada VM genere un identificador distinto al arrancar.

4.6. `parte2-kvm/guest/agr-net.sh` (en cada arranque de cada VM)

1. **Comprobación:** si falta `/etc/agr/node.conf`, termina con error.
2. **Rol:** levanta `lo` y lee el rol con `awk`.
3. **`dev_by_mac(mac)`:** recorre `/sys/class/net/*/address` y devuelve el nombre de la interfaz con esa MAC.
4. **Espera:** hasta 30 s a que aparezcan todas las MAC de `node.conf` (las tarjetas virtio pueden tardar en registrarse).
5. **Configuración de cada línea `iface`:**
    1. Encuentra la interfaz por MAC.
    2. Si su nombre no es el del plano, la baja y la renombra (`ip link set dev eth0 name ge0-0-5`).
    3. Borra sus direcciones, asigna la IP y la levanta.
    4. Lo anota en el registro (`journalctl -u agr-net`).
6. **Reenvío IP:** `ip_forward=1` en routers y `0` en PCs y servidor. Así un PC nunca encamina por accidente.
7. **Rutas:**
    - En un router con FRR y `frr.conf`, no hace nada: lo hará FRR.
    - En los demás, cada línea `route` se instala con `ip route replace` (`0.0.0.0/0` se escribe como `default`).

4.7. `parte2-kvm/guest/agr-net.service`

Directiva	Para qué
`DefaultDependencies=no`	Puede ejecutarse temprano, antes de la red
`After=systemd-udevd.service local-fs.target`	Después de que udev registre las tarjetas y de que el disco esté montado
`Before=network-pre.target network.target frr.service ssr-server.service`	Antes que la red, FRR y el servidor web: cuando FRR arranca, las interfaces ya tienen nombre e IP
`Wants=network-pre.target`	Hace que exista ese punto de sincronización
`Type=oneshot`, `RemainAfterExit=yes`	Se ejecuta una vez y queda "activo", que es lo que comprueba la espera del script
`WantedBy=multi-user.target`	Se lanza en el arranque normal

4.8. `parte2-kvm/guest/ssr-server.service`

- Arranca después de `agr-net` (`After` y `Wants`).
- Usuario `nodeapp`, directorio `/opt/ssr-master-server`, variables `NODE_ENV=production` y `PORT=3000`.
- `ExecStart=/opt/node/bin/node app.js` (por el fallo de `npm start` ya explicado).
- `Restart=on-failure` con 3 s de espera.

Sólo se habilita en SRV.

4.9. `parte2-kvm/guest/agr-info.sh` (al iniciar sesión)

Sólo en sesiones interactivas. Muestra:

- el nodo y su rol;
- las interfaces IPv4 (`ip -br -4 addr`);
- la tabla de rutas;
- según el rol:
  - **router:** el reenvío IP y si FRR está activo;
  - **servidor:** el estado de `ssr-server`;
  - **PC:** las órdenes para probar la web.

4.10. Ficheros generados por VM (ejemplo: RC)

- **`/etc/agr/node.conf`:** `role router` y tres líneas `iface` (las MAC `52:54:00:aa:03:00..02` con `ge0-0-5`, `ge0-0-1` y `ge0-0-2` y sus IPs), y `route 0.0.0.0/0 10.2.0.1`.
- **`/etc/frr/frr.conf`:** cabecera, tres `interface ... description ...` e `ip route 0.0.0.0/0 10.2.0.1`.
- **`/var/lib/libvirt/images/agr/agr-rc.xml`:** el XML del dominio.

Todos están, como referencia, en `doc/development/parte2/xml_generados/`.

4.11. `doc/build_pdf.sh`

1. Para cada `.md` de `doc/entregables/src/`, `pandoc` lo convierte en HTML autocontenido, con índice y con `estilo.css` y las imágenes incrustadas.
2. Si hay Chromium o Chrome, lo imprime a PDF en modo *headless*. Si no, usa `pandoc` con `xelatex`.
3. Los PDF quedan en `doc/entregables/`.

4.12. El servidor web SSR-master-server (código de terceros)

Ruta	Qué hace
`/`	Plantilla `index.ejs` (título “Data-Logger”)
`/record?id_nodo=&temperatura=&humedad=&co2=&volatiles=`	Añade una línea a `public/logs/<id_nodo>-<año>-<mes>-<día>.csv` (con cabecera si es nuevo) y responde `Saving: ...`
`/logs`	Lista de ficheros (`serve-index`) y descarga de los CSV

Fallo conocido: el doble `listen(3000)` en `app.js` y `bin/www` (por eso se lanza con `node app.js`). El mes del nombre del CSV empieza en 0, porque usa `getMonth()` de JavaScript.

5. Mapa del repositorio

```
parte1-vagrant/Vagrantfile          Parte 1 (§4.1)
parte2-kvm/agr_kvm.py               Parte 2: orquestador (§4.2)
parte2-kvm/topologia.json           Plan de numeración (§4.3)
parte2-kvm/plantilla_vm.xml         XML de Moodle (§4.4)
parte2-kvm/imagen/                  Imagen de Moodle (no versionada) + README
parte2-kvm/guest/build-base.sh      Construcción de la imagen base (§4.5)
parte2-kvm/guest/agr-net.sh|.service Red de cada VM (§4.6, §4.7)
parte2-kvm/guest/ssr-server.service Servidor web (§4.8)
parte2-kvm/guest/agr-info.sh        Información al iniciar sesión (§4.9)
doc/entregables/*.pdf               Memorias, manual, paso a paso y este documento
doc/entregables/src/                Fuentes Markdown, CSS y diagrama SVG
doc/development/enunciado/          Enunciado y plano del laboratorio
doc/development/parte1/             Medidas de la parte 1
doc/development/parte2/             XML de referencia, informe de pruebas, medidas
doc/build_pdf.sh                    Generación de los PDF (§4.11)
```